RESEARCH CONTINUATION

Four further Lean proof simplifications

for DiophantineConsistency

Weaker controller facts, direct homogeneous enumeration, shared Jones evaluation, and affine reflection

Prepared for Vladimir Reshetnikov

26 October 2026

Repository snapshot

`9f35798db4a63c25e435fca14d75931144657d7e`

https://github.com/VladimirReshetnikov/ProveIt

Result. Four source-specific proposals survive a screen against the 23 filed studies and five incoming archives. The clearest mathematical simplification is an affine reflection that makes the carrier soundness and completeness sign arguments instances of one integer lemma. The largest structural change replaces the Jones compiler's filtered Cartesian enumeration by the existing executable antidiagonal list. Two smaller changes share five duplicated finalizer proofs and remove unnecessary exact-order reasoning from four controller clients.

Evidence. The archive includes standalone Lean experiments, four candidate patches, a packet-by-packet exclusion ledger, exact source and archive hashes, measurements, and validation records. Standalone proof checks, static inspection, and incomplete integration checks are distinguished explicitly. No candidate is represented as an adopted repository change.

Novelty is relative to the specified proposal corpus. No claim of priority for the underlying mathematical identities is made.

Contents

1 What remains after the earlier proposals are assumed applied **2**
1.1 Four retained directions . . . 2
1.2 What is deliberately not proposed again . . . 2

2 A: share the existing Jones finalizer evaluation **3**
2.1 The duplication is exact and local . . . 3
2.2 Concrete edit . . . 3
2.3 Measured source effect and validation boundary . . . 3

3 B: one affine lemma, with soundness obtained by reflection **4**
3.1 The same relation appears in both directions . . . 4
3.2 The common integer lemma . . . 4
3.3 Completeness keeps its original one-sided hypothesis . . . 5
3.4 Soundness is the reflected instance . . . 5
3.5 Patch scope . . . 6

4 C: generate the Jones homogeneous star directly **6**
4.1 The current proof pays for a representation conversion . . . 6
4.2 Replace the source of the enumeration, not its order proof . . . 6
4.3 The structural difference is large . . . 7
4.4 Compatibility is a real choice . . . 7
4.5 Residual novelty after the baseline . . . 8

5 D: fixed controller clients need fixed powers, not least orders **8**
5.1 Separate sufficient equations from exact classification . . . 8
5.2 One fixed-point iteration lemma . . . 9
5.3 What must be retained . . . 9
5.4 The actual source payoff is small, the contract payoff is clear . . . 9

6 Evidence, measurements, and what has not been established **10**
6.1 Three levels of evidence . . . 10
6.2 Candidate patch measurements . . . 10
6.3 Necessary integration checks . . . 10

7 A tempting shortcut that should not be adopted **11**

8 Using the archive **11**

A Primary sources and reproducible references **12**

1 What remains after the earlier proposals are assumed applied

The question is a continuation task: a change is useful here only if it still removes work after the outcomes of the earlier proposals are granted. This is stricter than searching for long proof bodies in the current checkout. A long body may already have a proposed replacement; a theorem marked "deferred" may already supply exactly the abstraction one is about to suggest. I therefore treated adopted, pending, design-only, optional, and notes-only proposals alike as part of the assumed baseline.[1, 2]

The current source was pinned before analysis. All 686 Lean files under the requested maintained subtree were checked against their recorded Git blob hashes and byte sizes. The corresponding selected prior-artifact census contains 643 files, including the five incoming ZIPs; these also pass the byte-integrity check. These counts describe retrieved source coverage, not a claim that every proof was semantically reviewed. The maintained subtree contains 85,917 physical Lean lines; 403 files remain when the checks directory and files ending in `Checks.lean` are excluded. Exact counting rules are supplied in `audit/source_integrity.json`.

The semantic novelty screen read the packet index, adoption record, all 23 filed packet READMEs, all five incoming READMEs, principal report and section maps, explicit future-question sections, and relevant supplementary notes and prototype matches. Candidate-specific searches and source review were then used to decide whether each proposed mechanism was already present. The ledger records the limits: it does not assert a fresh semantic reading of every copied historical baseline line, every binary member, or every historical archive linked transitively from the large incoming README.

1.1 Four retained directions

ID	Surviving simplification	Main boundary
A	Share the five existing Jones finalizer evaluation lemmas across the two syntax constructors.	Proof-only edit; no expression data change.
B	Use one affine positivity lemma and reflection about $y = 1$ for carrier soundness and completeness.	Preserve the integral domain and one-sided completeness hypothesis.
C	Enumerate only degree-four exponent vectors using `List.Nat.antidiagonalTuple`.	Executable expression trees change.
D	Prove only the fixed-power and annihilation facts consumed by the fixed controller factors.	Auxiliary prime-power exact order still matters.

The recommended implementation order is A, B, D, then C. A has the smallest conceptual risk. B removes a repeated mathematical argument. D improves the sufficient contract of four clients, with little source-line reduction. C has the strongest effect on the generated syntax but requires a deliberate compatibility decision.

1.2 What is deliberately not proposed again

The earlier corpus already covers assignment naturality, witness transport, list lookup APIs, decoder fusion, scanner iteration, common residual sums, coefficient envelopes, circuit certificates, degree tactics, source-register equations, guarded zero-set transport, Pell sign and comparison lemmas, exact quotient recovery, internal arithmetic specification interfaces, one-hot support, cyclic carries, helical periods and band coordinates, shared Wang edges, receiver causality, and canonical tile-row recognition.[1, 2]

Generic advice to use `simp`, add helper records, shorten imports, profile elaboration, or generate repeti-

tive proofs is also established methodology rather than a new contribution. The included independent review rejected a fifth proposal because no distinct surviving client benefit could be established. Four concrete proposals are more informative than a longer list padded with old abstractions.

2 A: share the existing Jones finalizer evaluation

2.1 The duplication is exact and local

`JonesSyntax.lean` defines the evaluation of a `RingVars(Exprσ)` record and proves five private facts: `eval_rad`, `eval_wt`, `eval_dvsr`, `eval_dvnd`, and `eval_marg`. Later, `ExecutableJones.lean` imports that module, uses the same record evaluator, and proves the same five facts again. The second copy differs mainly in qualification and in writing application as a lambda or as `.eval`; these do not describe different semantics.[3, 4]

The two compilers differ in how they construct the upstream coding quantities. One uses coefficient readers on a polynomial; the other uses supplied executable syntax. That difference is handled by each module's own `eval_syntaxVars`. Once a `RingVars(Exprσ)` value has been supplied, its finalizer fields have identical meanings in both modules.

2.2 Concrete edit

Make the five existing theorems in `JonesSyntax` public, remove the second copies, and qualify the five uses in the executable compiler:

```
simp only [withInput, RefinedPair.value,
  Expr.eval_add, Expr.eval_sub, Expr.eval_mul,
  Expr.eval_pow, Expr.eval_var,
  JonesSyntax.eval_rad, JonesSyntax.eval_wt,
  JonesSyntax.eval_dvsr, JonesSyntax.eval_dvnd,
  JonesSyntax.eval_marg, eval_syntaxVars]
rfl
```

This is intentionally narrower than introducing a new generic evaluator or a semantic homomorphism framework. Those general ideas are already in the baseline. The new observation is that the needed concrete interface already exists in the direct import and is private for no demonstrated client requirement.

The patch makes no changes to `syntaxVars`, `withInput`, `expression`, `removeInput`, or the expression constructors. It adds no global simplification attributes. Consequently, this proposal by itself changes no literal expression tree, no coefficient order, and no compiler output data. It is independent of proposal C.

2.3 Measured source effect and validation boundary

The candidate changes `ExecutableJones.lean` from 300 to 251 physical lines and keeps `JonesSyntax.lean` at 179 lines: **49 lines removed, with no new helper proof**. This includes the qualified-use replacement. The archive contains the exact diff and its metric record, rather than an extrapolated repository-wide percentage.

The change was reviewed against both complete source blocks. A full fresh build of the complete Jones dependency closure was not established in this session. Therefore the patch is labelled a statically reviewed candidate, not a kernel-checked integrated replacement. Its next gate is short and concrete: compile the changed `JonesSyntax` first, compile `ExecutableJones` against that new object, and check both existing `eval_expression` endpoints. Reusing an old private-theorem object would not test the intended interface change.

3 B: one affine lemma, with soundness obtained by reflection

3.1 The same relation appears in both directions

The private carrier coordinate in `ThirteenFreshSeed.carrier_exists` and the recovered transport in `ThirteenCarrierSoundness.transport_nonneg` are related by

$$y = 1 + 4\left(b(m - V^2G) - G\right) = 1 + 4\left(bm - (1 + bV^2)G\right). \tag{1}$$

The completeness proof constructs m and proves $m > 0$ using several successive coefficient lower bounds. The soundness proof assumes $m > 0$, supposes $G < 0$, and develops a separate contradiction with the small-ordinate bound.[5, 6]

The surviving simplification is to use the affine symmetry of (1). No Pell recurrence, norm-minimality result, signed congruence calculus, or rational-root descent is needed for this local step. Those earlier themes solve different obligations.

3.2 The common integer lemma

Proposition 1 (Affine carrier positivity)**.** *Let $b, G, V, y, m \in \mathbb{Z}$. If $b > 0$, $G > 0$, $-2V < y$, and (1) holds, then $m > 0$. There is no sign assumption on V.*

Proof. Suppose $m \leq 0$. Since the variables are integers, $b \geq 1$ and $G \geq 1$. Put $A = 1 + bV^2$, which is positive. Then $bm \leq 0$ and $AG \geq A$, so

$$y = 1 + 4(bm - AG) \leq 1 - 4A = -3 - 4bV^2.$$

For every integer V and every $b \geq 1$,

$$4(4bV^2 - 2V + 3) = (4V - 1)^2 + 16(b - 1)V^2 + 11 > 0. \tag{2}$$

Thus $-3 - 4bV^2 < -2V$, contradicting $-2V < y$. □

The completed-square identity explains why the old proof's positive-V comparison $V \leq V^2$ is dispensable. It also makes the domain requirements visible. Integer discreteness supplies $b \geq 1$ and $G \geq 1$ from strict positivity. One cannot silently replace the integer variables by arbitrary positive real numbers while retaining the same argument.

The checked Lean core expresses the required inequalities directly:

```
theorem affine_carrier_pos {b G V y m : ℤ}
    (hb : 0 < b) (hG : 0 < G) (hsmall : -2 * V < y)
    (hy : y = 1 + 4 * (b * (m - V ^ 2 * G) - G)) :
    0 < m := by
  by_contra hm
  have hbm := mul_nonpos_of_nonneg_of_nonpos hb.le
    (show m ≤ 0 by omega)
  have hAG := mul_nonneg
    (show 0 ≤ 1 + b * V ^ 2 by positivity)
    (show 0 ≤ G - 1 by omega)
  have hbV := mul_nonneg
    (show 0 ≤ b - 1 by omega) (sq_nonneg V)
  nlinarith only [hy, hsmall, hbm, hAG, hbV,
    sq_nonneg (4 * V - 1)]
```

3.3 Completeness keeps its original one-sided hypothesis

The existing divisibility premises produce integers g, e with

$$G = bg, \qquad y - 1 = 4be.$$

Define the same explicit witness as in the current proof:

$$m = e + (1 + bV^2)g. \tag{3}$$

Substitution verifies (1); the common lemma proves $m > 0$. The new proof does not need to establish $g > 0$ separately or build the two coefficient comparison chains. More importantly, it does not strengthen the existing premise $-2V < y$ to the symmetric premise $|y| < 2|V|$.

The existing public parameter $0 < V$ can remain for compatibility, even though the shorter local proof does not use it. The standalone compilation reports that unused-parameter warning; it is not a proof failure. Removing the public premise is a separate API choice, not a condition of this proposal.

3.4 Soundness is the reflected instance

The transformation

$$(m, G, y) \longmapsto (-m, -G, 2 - y) \tag{4}$$

preserves (1). The reflection is around $y = 1$: the correct new ordinate is $2 - y$, not $-y$. Also replace V by $|V|$, which preserves its square.

Suppose the soundness hypotheses hold: $b > 0$, $m > 0$, and $|y| < 2|V|$. If $G < 0$, then $-G > 0$. Since $y \le |y| < 2|V|$, we have

$$-2|V| < 2 - y.$$

Apply the common positivity lemma to the reflected data. It gives $-m > 0$, contradicting $m > 0$. Hence $G \ge 0$.

This proof uses only an upper bound on the original y to form the reflected lower bound. It introduces neither a positivity assumption on the original V nor a new absolute-value condition in completeness.

3.5 Patch scope

The patch places the helper in `ThirteenCarrierSoundness`, which `ThirteenFreshSeed` already imports. It replaces both local arguments while preserving their existing theorem signatures. In the final documented patch, the soundness module grows from 290 to 297 lines, while the seed module shrinks from 217 to 197: **13 net lines removed, including the helper and its documentation**.

The reduction is modest in line count but substantial in proof structure. The two directions now share one reason that an incorrectly signed carrier coordinate is too large in magnitude. The exact witness (3), the outer native data, the Pell construction, and the polynomial definitions are unchanged.

The standalone core, soundness adapter, and completeness adapter compile under Lean 4.32.0. Their axiom output is limited to the standard `propext`, `Classical.choice`, and `Quot.sound`. Separate patched-module elaboration attempts and their dependency limitations are recorded in the validation artifacts; they are not promoted to a clean build of the entire pinned project.

4 C: generate the Jones homogeneous star directly

4.1 The current proof pays for a representation conversion

The executable Jones compiler currently enumerates all functions

$$k : \mathrm{Fin}(\nu + 2) \to \mathrm{Fin}(5),$$

then tests whether their coordinate sum is four. Valid vectors produce a term; the others produce a literal zero expression. A recursive `eval_vectorSum` theorem proves the Cartesian enumeration, and `sum_bounded_eq_star` constructs the conversion to natural-valued vectors in the homogeneous star. The latter proof must show membership, injectivity, and surjectivity, including the lift of every natural coordinate to `Fin5`.[4]

The target set is already

$$\mathrm{star}(\nu, 4) = \left\{ k : \mathrm{Fin}(\nu + 2) \to \mathbb{N} : \sum_j k_j = 4 \right\}.$$

In the underlying Jones formalization it is defined as `Finset.univ.piAntidiag4`. The pinned Mathlib provides an executable list of precisely these tuples, its membership characterization, and its no-duplicates theorem. It also provides the equality between the finite antidiagonal and `piAntidiag`.[7, 8, 9]

4.2 Replace the source of the enumeration, not its order proof

The new definition is simply

```
def starSum {σ : Type*} (ν : ℕ)
    (f : (Fin (ν + 2) → ℕ) → Expr σ) : Expr σ :=
  QuarticSyntax.sumExpressions
    ((List.Nat.antidiagonalTuple (ν + 2) 4).map f)
```

An explicit proof can turn the duplicate-free list sum into a finite-set sum and identify membership on both sides. The pinned library permits a still shorter adapter: after the existing sum evaluator and the supplied antidiagonal equality are rewritten, the remaining equality is definitional.

```
rw [starSum, eval_sumExpressions, List.map_map, star,
  Finset.piAntidiag_univ_fin_eq_antidiagonalTuple]
rfl
```

The direct list-to-finite-set equality was separately checked in Lean. The complete proposed Jones module is still subject to the integration gate below.

The independent checked prototype proves this for every additive commutative monoid, every coordinate count n, every degree d, and any callback f:

$$\sum_{k \in \mathrm{List.Nat.antidiagonalTuple}(n,d)} f(k) = \sum_{k \in \mathrm{univ.piAntidiag}(d)} f(k). \tag{5}$$

It also supplies an evaluation adapter parameterized by a syntax sum and its existing evaluation theorem. This isolates the mathematical change from the long Jones dependency closure.

There is no need to invent a new enumerator or reprove its recursive combinatorics. In particular, do not use `Finset.toList` or `Multiset.toList` as the executable implementation: these interfaces are noncomputable in the pinned library. The list antidiagonal is the appropriate constructive object. Its construction splits the remaining total degree among the remaining positions.

4.3 The structural difference is large

For $n = \nu + 2 > 0$, the old Cartesian enumeration visits 5^n candidate vectors. The homogeneous list contains

$$\binom{n+3}{4} \tag{6}$$

valid degree-four vectors, by the usual stars-and-bars count. The old syntax contains rejected zero branches; the new outer list contains only the valid callbacks, followed by the sum constructor's terminal identity.

Coordinates n	Old candidate vectors 5^n	Valid list entries $\binom{n+3}{4}$
2	25	5
4	625	35
8	390,625	330
10	9,765,625	715
16	152,587,890,625	3,876

These are exact combinatorial counts, not measured elapsed times. The implementation also allocates tuple representations and traverses internal recursive branches. A naive recursion down to zero remaining coordinates can visit more leaves than the number of valid outputs. The report therefore does not identify (6) with total primitive operations, nor does it claim a benchmark speedup. The compelling proof-maintenance point is that the Cartesian/filter conversion and its coordinate-bound obligations disappear.

4.4 Compatibility is a real choice

This proposal changes `starSum`'s expression tree. Evaluated integer values and polynomial semantics can be preserved by (5); raw syntax identity, tree hashes, generated gate counts, and every definitional

equality do not follow from that theorem. Existing compiler artifacts or regressions that depend on literal syntax must be reviewed or regenerated deliberately.

The proposed cleanup removes the public helpers `vectorSum`, `eval_vectorSum`, and `sum_bounded_eq_star`. A textual reference scan of the complete maintained subtree finds all uses of those helpers confined to `ExecutableJones.lean`. The same holds in the selected historical Lean sources that were available locally. This is not a search of every possible external consumer or every unextracted repository path. Keeping the old helper API as a compatibility layer is possible, but then its source lines cannot honestly be counted as deleted.

The candidate implementation changes `ExecutableJones.lean` from 300 to 258 lines: **42 lines removed** under the explicit cleanup choice. That count is separate from A's 49-line removal. The two diffs touch different blocks and are supplied independently, so the data-preserving change can be adopted without accepting the executable change.

4.5 Residual novelty after the baseline

Earlier sparse coefficient and circuit proposals concern fixed catalogs, coefficient normalization, or interpreting supplied rows. They do not remove the bounded-vector representation at this particular homogeneous-index interface. Even if their APIs are already present, this compiler still needlessly proves that a filtered Cartesian product denotes an existing constructive antidiagonal. The new contribution is the recognition and removal of that representation conversion.

5 D: fixed controller clients need fixed powers, not least orders

5.1 Separate sufficient equations from exact classification

The maintained controller uses

$$B = 2^{94517}, \qquad p = 120199, \qquad \text{dimension}(n) = p^n.$$

Its checked residues include

$$B = 37 \text{ in } \mathbb{Z}/47\mathbb{Z}, \qquad B = 1283 \text{ in } \mathbb{Z}/2011\mathbb{Z}.$$

The data module proves exact orders of both 2 and B at the two fixed primes. Four production clients use the exact B orders: two dimension-power reductions and two generator-power identities. Inspection of their conclusions shows that none asks for a least exponent.[10, 11, 12]

One route keeps the existing checked equations $2^{23} = 1$ modulo 47 and $2^{402} = 1$ modulo 2011. Power commutation gives the corresponding equations for $B = 2^{94517}$. Exponent congruence then proves the downstream statements without any proof that 23 or 402 is minimal.

For the literal controller there is an even shorter route. Check the needed fixed residues directly at exponent p and at exponent $p - 1$:

$$\begin{aligned} 37^{120199} &= 37 && \text{in } \mathbb{Z}/47\mathbb{Z}, \\ 1283^{120199} &= 1283 && \text{in } \mathbb{Z}/2011\mathbb{Z}, \\ 37^{120198} &= 1 && \text{in } \mathbb{Z}/47\mathbb{Z}, \\ 1283^{120198} &= 1 && \text{in } \mathbb{Z}/2011\mathbb{Z}. \end{aligned}$$

The supplied prototype checks all four with `reduce_mod_char`. It does not materialize the huge natural number B.

5.2 One fixed-point iteration lemma

Lemma 1. *In any monoid, if $x^p = x$, then $x^{p^n} = x$ for every $n \in \mathbb{N}$.*

Proof. At $n = 0$ the exponent is one. For the induction step,

$$x^{p^{n+1}} = (x^{p^n})^p = x^p = x.$$

□

```
theorem pow_power_fixed {M : Type*} [Monoid M]
    {x : M} {p : ℕ} (h : x ^ p = x) (n : ℕ) :
    x ^ (p ^ n) = x := by
  induction n with
  | zero => simp
  | succ n ih => rw [pow_succ, pow_mul, ih, h]
```

No primality, finite-group enumeration, coprimality of an exponent, order-minimality result, or positivity of n is needed for this lemma. The case $n = 0$ is included and is important because the current independent dimensions allow zero exponents.

After rewriting the residue of B, each fixed-prime `power_dimension` theorem becomes an application of this lemma. Each fixed-prime `generator_cast` theorem becomes the already checked annihilation identity at $p-1$. The auxiliary-field dimension theorem can continue using its existing Frobenius proof; replacing a good one-line proof merely for visual uniformity would not help.

5.3 What must be retained

There are two separate limits to the simplification.

First, the exact fixed-prime order declarations are public and are exercised by `CurrentControllerData2011Checks`. The supplied patch leaves those declarations and their checks intact. It changes the sufficient dependencies of the four production proofs, not the truth or availability of stronger results.

Second, the auxiliary prime-power exact order is genuinely used later:

$$\texttt{generator_order_succ} \longrightarrow \texttt{full_order},$$
$$\texttt{full_order} \longrightarrow \texttt{block_order} \longrightarrow \texttt{exists_coordinates}.$$

That chain constructs a coordinate bijection with the required full orbit size. An arbitrary annihilating exponent would not establish the necessary injectivity and cardinality. It must not be weakened along with the four fixed-factor identities.[13, 12]

There is also a distinction between a theorem dependency and a compilation dependency. If exact-order proofs remain in an imported module, Lean still compiles that module. A subsequent module split could put optional exact-order certificates behind a compatibility facade, but that is a separate architectural change and is not counted as an achieved speedup here.

5.4 The actual source payoff is small, the contract payoff is clear

The candidate diff retains every exact-order theorem and adds the generic iterator only once. Its two touched modules change from 306 to 305 lines in total: **one line removed**. This proposal is therefore recommended for the clearer and weaker proof obligation, not for a dramatic source reduction.

The standalone experiment verifies the general monoid lemma, all four literal residue facts, and transport from an arbitrary natural B satisfying the two recorded residues. Full integration of the edited controller modules was not claimed. In particular, the standalone tests do not certify the larger auxiliary orbit construction; that construction is intentionally outside this change.

6 Evidence, measurements, and what has not been established

6.1 Three levels of evidence

The archive separates three kinds of claim:

1. **Pinned source facts.** Source blobs, current theorem statements, repeated proof blocks, identifier references, and patch line counts are reproducible from the recorded commit and supplied scripts.
2. **Standalone Lean results.** The mathematical controller core, constructive-enumeration bridge, and affine reflection are compiled with the pinned Lean 4.32.0 toolchain and the pinned Mathlib dependency version. Full output, including axiom reports and nonfatal warnings, is included.
3. **Repository integration.** The diffs are proposals. A complete clean build of the entire pinned ProveIt dependency closure and a complete before/after axiom audit of all downstream theorems have not been established. Any cached-dependency module elaboration is labelled separately in its receipt.

The final standalone logs, not an earlier failed editing attempt, determine the reported success. No `sorry`, `admit`, `sorryAx`, or `native_decide` is used as a proof shortcut in the successful core deliverables. The axiom reports expose the standard Lean axioms actually present; they do not assert axiom-free proofs.

6.2 Candidate patch measurements

ID	Modules	Before	After	Net reduction
A	JonesSyntax; ExecutableJones	479	430	49 lines
B	ThirteenCarrierSoundness; ThirteenFreshSeed	507	494	13 lines
C	ExecutableJones, as an independent alternative	300	258	42 lines
D	CurrentControllerUnits2011; CurrentPowerCoordinates2011	306	305	1 line

Counts are physical source lines, including new helper code and documentation, measured on the final candidate files. They are not counts of kernel proof terms, compiled object size, or elapsed-time improvements. A and C are independent edits to the same module; their rows must not be treated as separate copies of the module when computing a combined total. No percentage reduction across the entire project is extrapolated from these examples.

6.3 Necessary integration checks

For A, rebuild the exported theorem module before its consumer and verify both existing expression-evaluation endpoints. For B, rebuild both modules and their soundness/completeness clients against the actual current prerequisites; compare old and new public theorem types and check that the explicit witness is unchanged. For D, compare the dependencies of the four selected theorems while retaining

the auxiliary-order chain and public exact-order checks. For C, verify evaluated and polynomial equality for the new syntax and explicitly review every assumption about raw trees, generated sizes, hashes, and compatibility helpers.

These are remaining acceptance conditions, not an invitation to require a large unrelated test campaign. Once those concrete risks are resolved, there is no reason to repeat every historical finite-certificate computation to adopt a local proof-only change.

7 A tempting shortcut that should not be adopted

The review also considered whether a bound-code observation certificate could imply formula-code validity, thereby removing the explicit `IsFormCodeSem` guard from a uniqueness theorem. This would be a new shortcut relative to the screened corpus, but it is not justified by the existing interfaces.

The formula-validity predicate is defined through the least constructor-closed interpretation. The bound-certificate relation supplies local postfixed observation equations. Projecting a locally closed graph does not identify it with the least generated domain. Quantifier rows also preserve the bound, so the numeric bound is not a strictly decreasing rank that would close the gap. The relevant internal `ZFAxioms` interface deliberately lacks Foundation/Regularity. An external natural-number induction on arbitrary internal codes would not repair that issue.[14, 15]

This failed direction is included because it illustrates the standard used for the four positive proposals: absence from the earlier corpus is not enough. A recommendation must also remove a genuine current obligation without quietly strengthening the assumptions or weakening the intended theorem.

8 Using the archive

The report is accompanied by:

- `lean/`: four standalone proof files and their final validation output;
- `patches/`: four independent candidate diffs against the pinned source;
- `audit/`: source integrity, the complete packet exclusion ledger, incoming archive hashes, independent semantic review, exact target references, metrics, and validation scope;
- `scripts/`: commands to rebuild the report, replay the standalone checks in a Lean 4.32.0 project, and inspect the patch targets;
- the repository's `lean-toolchain` and `lake-manifest.json` as version evidence.

To replay the Lean cores, use an existing checkout with the recorded toolchain and dependencies and run the supplied script with that checkout as its argument. The script compiles the four standalone sources; it does not silently apply patches or infer that an integrated build passed. The report can be rebuilt with XeLaTeX from its own directory.

The practical next action is to land A and B as separate proof-only changes after their focused integration gates, then evaluate D for its contract simplification. C should be reviewed as an executable compiler refactor with an explicit syntax-compatibility decision. Each change addresses work still present after the earlier proposal outcomes are granted.

A Primary sources and reproducible references

All ProveIt links below are pinned to the reviewed commit. The machine-readable source-reference map includes exact line ranges, and the source-integrity receipt verifies the local bytes. Mathlib references are pinned to the dependency commit in `lake-manifest.json`, not to the current website's default branch.

References

[1] ProveIt, *Lean proof simplification research, including ADOPTION.md and packets a–w*. https://github.com/VladimirReshetnikov/ProveIt/tree/9f35798db4a63c25e435fca14d75931144657d7e/Computability/DiophantineConsistency/research/lean-proof-simplification.

[2] ProveIt, *docs/incoming*, five current proposal archives and index. https://github.com/VladimirReshetnikov/ProveIt/tree/9f35798db4a63c25e435fca14d75931144657d7e/docs/incoming.

[3] ProveIt, *JonesSyntax.lean*. https://github.com/VladimirReshetnikov/ProveIt/blob/9f35798db4a63c25e435fca14d75931144657d7e/Computability/DiophantineConsistency/Lean/DiophantineConsistency/JonesSyntax.lean.

[4] ProveIt, *ExecutableJones.lean*. https://github.com/VladimirReshetnikov/ProveIt/blob/9f35798db4a63c25e435fca14d75931144657d7e/Computability/DiophantineConsistency/Lean/DiophantineConsistency/ExecutableJones.lean.

[5] ProveIt, *ThirteenFreshSeed.lean*. https://github.com/VladimirReshetnikov/ProveIt/blob/9f35798db4a63c25e435fca14d75931144657d7e/Computability/DiophantineConsistency/Lean/DiophantineConsistency/ThirteenFreshSeed.lean.

[6] ProveIt, *ThirteenCarrierSoundness.lean*. https://github.com/VladimirReshetnikov/ProveIt/blob/9f35798db4a63c25e435fca14d75931144657d7e/Computability/DiophantineConsistency/Lean/DiophantineConsistency/ThirteenCarrierSoundness.lean.

[7] ProveIt, *Paper1982/Coding.lean*, homogeneous star definition. https://github.com/VladimirReshetnikov/ProveIt/blob/9f35798db4a63c25e435fca14d75931144657d7e/Computability/HilbertTenthProblem/Lean/Diophantine/Paper1982/Coding.lean.

[8] Mathlib, *Data/Fin/Tuple/NatAntidiagonal.lean*, executable lists and no-duplicates theorem. https://github.com/leanprover-community/mathlib4/blob/81a5d257c8e410db227a6665ed08f64fea08e997/Mathlib/Data/Fin/Tuple/NatAntidiagonal.lean.

[9] Mathlib, *Algebra/Order/Antidiag/Pi.lean*, finite antidiagonal comparison. https://github.com/leanprover-community/mathlib4/blob/81a5d257c8e410db227a6665ed08f64fea08e997/Mathlib/Algebra/Order/Antidiag/Pi.lean.

[10] ProveIt, *CurrentControllerData2011.lean*. https://github.com/VladimirReshetnikov/ProveIt/blob/9f35798db4a63c25e435fca14d75931144657d7e/Computability/DiophantineConsistency/Lean/DiophantineConsistency/CurrentControllerData2011.lean.

[11] ProveIt, *CurrentControllerUnits2011.lean*. https://github.com/VladimirReshetnikov/ProveIt/blob/9f35798db4a63c25e435fca14d75931144657d7e/Computability/DiophantineConsistency/Lean/DiophantineConsistency/CurrentControllerUnits2011.lean.

[12] ProveIt, *CurrentPowerCoordinates2011.lean*. https://github.com/VladimirReshetnikov/ProveIt/blob/9f35798db4a63c25e435fca14d75931144657d7e/Computability/DiophantineConsistency/Lean/DiophantineConsistency/CurrentPowerCoordinates2011.lean.

[13] ProveIt, *CurrentAuxiliaryOrder2011.lean*. https://github.com/VladimirReshetnikov/ProveIt/blob/9f35798db4a63c25e435fca14d75931144657d7e/Computability/DiophantineConsistency/Lean/DiophantineConsistency/CurrentAuxiliaryOrder2011.lean.

[14] ProveIt, *ObjectCodeBound.lean*. https://github.com/VladimirReshetnikov/ProveIt/blob/9f35798db4a63c25e435fca14d75931144657d7e/Computability/DiophantineConsistency/Lean/DiophantineConsistency/ObjectCodeBound.lean.

[15] ProveIt, *FormulaClosedSet.lean*. https://github.com/VladimirReshetnikov/ProveIt/blob/9f35798db4a63c25e435fca14d75931144657d7e/SetTheory/BoundedConsistency/Lean/BoundedZFCConsistency/FormulaClosedSet.lean.